
Dawid-Skene on the Server

Proposed Changes and Controlled Experiment Plan

A plain-English guide

Document status: design and feasibility plan only.

Main conclusion: the change is technically feasible as a server-only experiment, but it has not yet been implemented, validated, or approved as the default.

Change boundary: clients keep doing exactly what they do today. Dawid-Skene is added only where the server combines the clients' hard-label proposals.

Branch: dawid-skene

Prepared: 11 August 2026

Audience: researchers, engineers, reviewers, and non-specialist stakeholders

Contents

1	Executive summary	1
2	The current workflow	1
3	Exactly what changes	2
4	How Dawid-Skene works, without the jargon	5
5	Input rules and deterministic behavior	5
6	Safety, fallback, and shadow mode	6
7	Software changes	6
8	Configuration and starting values	7
9	Feasibility and expected cost	8
10	The controlled experiment	9
11	What will be measured	10
12	Decision gates	11
13	Privacy, audit, and reproducibility	12
14	Risks and limitations	13
15	Verification plan	14
16	Implementation sequence	15
17	Decisions and recommended defaults	15
A	Plain-English server pseudocode	16
B	The 12-setting shadow screen	17
C	Glossary	17
D	Reference and scope note	18

1 Executive summary

Today, every client gives the server a proposed class label for each sample in a shared open dataset. A client can also abstain when it is not confident that a sample is familiar. The server currently combines the submitted labels by counting votes.

The proposed change replaces only that vote-counting decision with Dawid-Skene server aggregation. Dawid-Skene does more than count answers. It repeatedly estimates two things together:

- the most likely true class of each shared sample; and
- the class-specific mistake pattern of each client that supplied enough labels.

This can help when clients have different strengths. For example, one client may distinguish two traffic classes well while another often confuses them. Majority voting treats both opinions as equally informative. Dawid-Skene can learn their different confusion patterns from the label table and use those patterns when estimating the final label.

The most important boundary

Client training, client prediction, the discriminator, filtering, hard-label upload, distillation, the dataset, and the network payload remain unchanged. The new calculation is entirely inside the server's hard-label aggregation step. Supporting server configuration, diagnostics, reports, audits, and tests must also be added so the change can be run safely.

The recommended first use is **shadow mode**. In shadow mode, the server computes Dawid-Skene and records what it would have decided, but still broadcasts the existing majority result. This gives a safe comparison on exactly the same client submissions. Active use should begin only after mathematical, accuracy, latency, privacy, and failure-handling gates pass.

This is a proposed adaptation of the Dawid-Skene idea, not a reproduction of the complete FedDS method in the reference paper. The paper uses reliability estimates to weight uploaded model parameters. This repository does not upload client models in this workflow. Here, Dawid-Skene is used only to combine the hard pseudo-labels that clients already upload.

2 The current workflow

One current federated round can be understood in seven steps:

1. Each client trains its persistent classifier on its own labelled data.
2. The classifier predicts classes for the same shared open dataset.
3. The client's discriminator decides which samples appear familiar enough to use.
4. The client uploads a hard-label vector. Legal class values are 0 through 10. The value -1 means "I abstain for this sample."
5. The server checks the submissions and combines the labels by deterministic majority vote.
6. The server broadcasts one `global_labels` vector and one `valid_mask` vector.
7. Existing client distillation and server-model training consume those two arrays.

All clients label the same ordered list of 8,900 open samples. This common ordering is what makes a client-by-sample annotation table possible on the server.

3 Exactly what changes

Table 1: Current and proposed server behavior.

Area	Current behavior	Proposed behavior
Label aggregation	Count each accepted client label equally and choose the lowest class index if there is an exact tie.	Estimate a class prior, one class-specific confusion matrix per eligible client, and a posterior class probability per sample; then choose the most likely class.
Operating modes	Majority only.	Majority, shadow Dawid-Skene, or active Dawid-Skene. Majority remains the software default.
Failure behavior	Use the majority result produced by the current path.	Return visibly and deterministically to that majority result if Dawid-Skene lacks support, becomes numerically invalid, or does not converge under the locked policy.
Diagnostics	Vote totals, participation, labels, and masks.	Add convergence, iterations, objective history, raw likelihood, timing, disagreement, fallback reason, support, and restricted client-confusion summaries.
Configuration	No Dawid-Skene fields.	Add explicit, validated options for initialization, smoothing, evidence thresholds, stopping, numerical safety, fallback, and audit storage.
Reporting	Existing experiment identity.	Keep extension runs separate by run kind and aggregation mode so they cannot replace a canonical report cell.
Testing	Existing majority and protocol tests.	Add mathematical, missing-data, determinism, failure, privacy, resume, reporting, and performance tests.

3.1 What does not change

- No client model parameters are uploaded.
- No client-side Dawid-Skene calculation is added.
- Clients keep training, predicting, filtering, and uploading the same hard-label vector.
- -1 remains an abstention. It does not become a twelfth class.
- The uplink and downlink keep the same array names, shapes, data types, and logical byte counts. Serialized message sizes are measured separately because values can change.
- The server still broadcasts only `global_labels` and `valid_mask`. It never broadcasts posterior probabilities, confusion matrices, or reliability scores.

- Existing distillation and server-model training continue after the aggregation decision.
- The dataset, partitions, CNN backbone, optimizer, learning rate, local epochs, batch size, discriminator setting, and primary round count remain locked between experimental arms.
- True open-set labels remain unavailable to live aggregation and training.

3.2 Notation used in the architecture

The detailed figure on the next page uses the following short names:

Symbol	Plain-English meaning
J	Number of accepted clients in the current server aggregation.
N	Number of shared open samples; currently 8,900.
C	Number of classes; currently 11.
Y	The client-by-sample table of submitted labels.
Q	The estimated probability of every candidate class for every usable sample.
M_j	Client j 's estimated class-specific confusion matrix for this round.
π	Estimated class proportions for the usable samples.
-1	Missing observation because the client abstained; never a class.

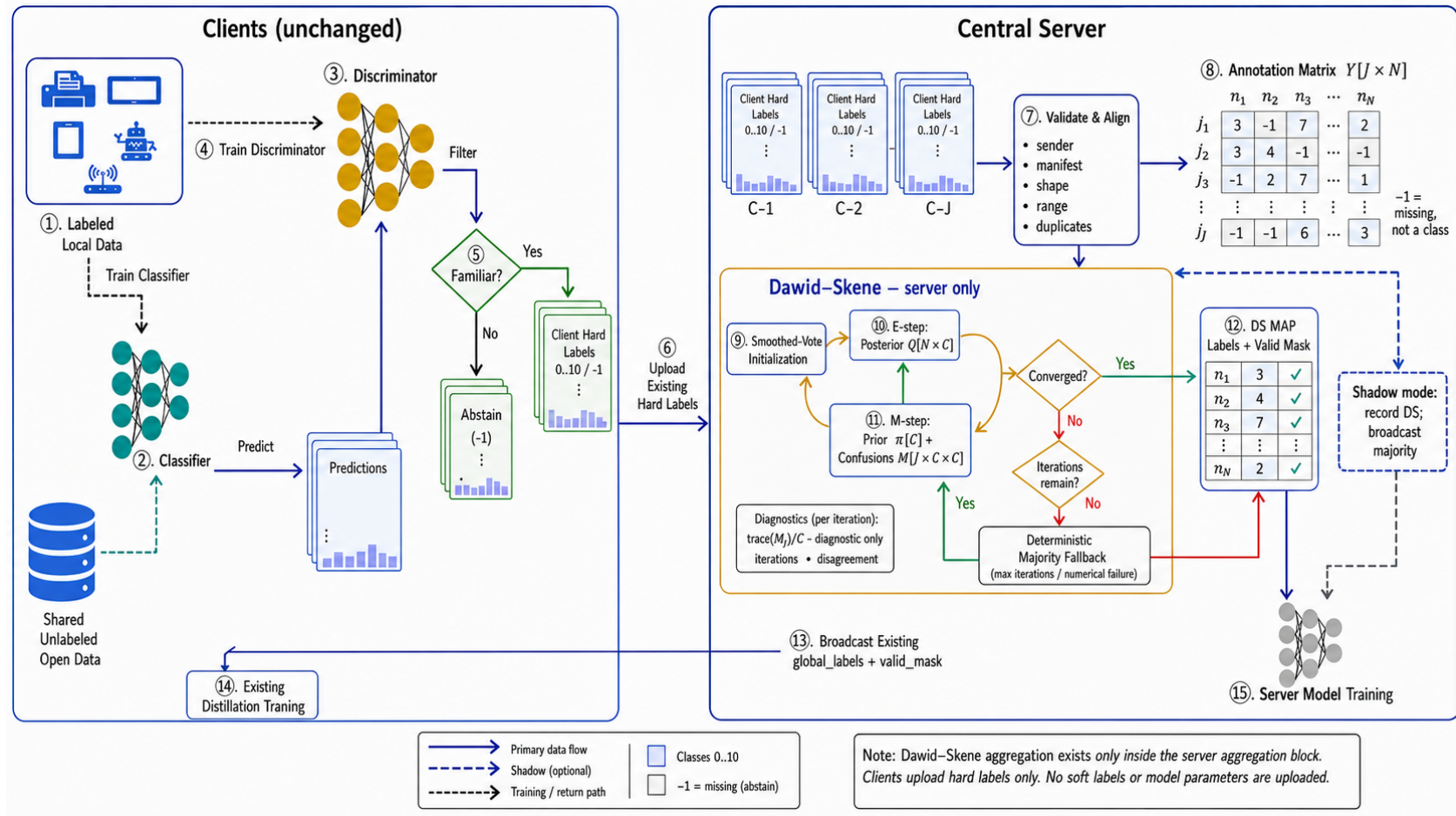


Figure 1: Detailed architecture. Clients are unchanged. Dawid-Skene exists only inside the central server aggregation block. In active mode, a successful fit supplies the hard MAP labels. In shadow mode, the server records the Dawid-Skene result but broadcasts majority labels. If the iteration limit is reached without convergence, or a numerical check fails, the majority fallback is used. Downstream training is not part of the Dawid-Skene loop.

4 How Dawid-Skene works, without the jargon

Majority voting asks only, “How many clients chose each answer?” Dawid-Skene also asks, “Based on the complete pattern of agreements and disagreements, which clients are informative for this particular class?”

The server performs the following cycle independently in each round:

1. **Build the table.** Put every accepted client’s label vector into a stable client-by-sample matrix.
2. **Ignore abstentions.** A -1 entry contributes no label evidence for that client and sample.
3. **Start cautiously.** Use smoothed vote proportions as the initial estimate. Smoothing avoids declaring any class mathematically impossible at the start.
4. **Reconsider the samples.** Estimate the probability that each sample belongs to each class, using the current client confusion patterns. This is the E-step.
5. **Reconsider the clients.** Estimate, for every eligible client and true class, how that client tends to label the class. This is the M-step.
6. **Repeat safely.** Alternate the previous two steps until the regularized objective changes by no more than the tolerance, or the iteration limit is reached.
7. **Choose hard labels.** For every supported sample, select the class with the largest posterior probability. Preserve the existing validity-mask interface.

The estimated scalar

$$\text{diagnostic reliability of client } j = \frac{\text{trace}(M_j)}{C}$$

is stored only as a restricted diagnostic. It does not weight uploaded model parameters and it is not sent to clients.

4.1 Important technical safeguards in simple terms

- Calculations use 64-bit floating-point numbers and log space. This avoids tiny probability products becoming zero because of computer precision.
- Small pseudocounts are added to class and confusion counts. This prevents unsupported rows from becoming impossible zero-probability rows.
- Because pseudocount priors are active, convergence is checked using a matching regularized objective. Raw data likelihood is recorded separately rather than used under the wrong name.
- Vote-based initialization encourages the hidden class numbers to retain the repository’s class meanings. It does not mathematically guarantee that label switching or a poor local solution can never occur.
- Confusion matrices are estimated again in every round in the primary study. They are not permanent client reputation scores. Warm start is disabled.

5 Input rules and deterministic behavior

The server must produce the same answer when the same valid submissions arrive in a different order. The adapter therefore applies explicit rules:

- Every label vector must have exactly N entries and contain only -1 or classes 0 through $C - 1$.
- Sender identifiers are processed in sorted order.
- Identical retries from one sender are accepted once and recorded as retries.
- If the same sender submits conflicting label vectors, that sender is rejected for the round and the reason is recorded.
- Clients below the minimum annotation count are excluded from estimation and reported.
- The proposed practical support gate requires at least three eligible clients. This is a guard, not a proof that every three-client problem is statistically identifiable.
- Samples with too few usable observations are not fitted or broadcast as valid.
- An exact class tie uses the lowest class index, preserving deterministic behavior.

6 Safety, fallback, and shadow mode

6.1 When the server falls back to majority

The proposed policy is `majority_on_failure_or_nonconvergence`. The server records the reason and uses deterministic majority voting if any of the following occurs:

- there are no estimable observations;
- fewer than the required number of clients are eligible;
- a prior, posterior, confusion value, likelihood, or objective becomes undefined or infinite;
- a probability vector does not normalize correctly;
- the regularized objective decreases beyond the numerical tolerance; or
- the maximum iteration count is reached without convergence.

Fallback keeps a round running; it does not prove that the majority answer is correct. The fallback rate is therefore a reported outcome and a decision gate.

If no valid proposal is received at all, returning “no update” is unsafe because an orchestrator can retain old arrays. The server must instead emit an explicit vector of -1 labels and an all-false validity mask. This prevents stale labels from being reused.

6.2 Why shadow mode comes first

In shadow mode the server computes and audits Dawid-Skene, but broadcasts majority labels. The training trajectory therefore stays on the current majority path. This is the only live mode where majority and Dawid-Skene see exactly the same client proposals throughout the run.

Once active Dawid-Skene labels are broadcast, client models can develop differently from the majority arm. Later proposals are then part of the treatment effect; they should not be described as identical. Active comparisons instead share the same initial state, seeds, scenario, code, hardware controls, and paired run identity.

7 Software changes

Table 2: Planned implementation points.

File or component	Plain-English responsibility
<code>src/ssfl/protocols/dawid_skene.py</code>	New self-contained NumPy estimator. It owns the E-step, M-step, initialization, stopping rules, invariants, result type, and failure signal.
<code>src/ssfl/protocols/ssfl.py</code>	Turn validated hard-label proposals into the stable annotation matrix, call the selected aggregator, and return the existing aggregation result fields.
<code>src/ssfl/config.py</code>	Add resolved Dawid-Skene fields, ranges, valid mode combinations, stable configuration hashing, and rejection of unsupported warm start.
<code>src/ssfl/strategies/ssfl.py</code>	Select majority, shadow, or active Dawid-Skene inside <code>aggregate_train</code> ; guarantee explicit no-proposal output and visible fallback.
<code>src/ssfl/server_app.py</code>	Build validated options and inject a diagnostics ledger into the strategy. Client and distillation wiring stays unchanged.
<code>src/ssfl/metrics.py</code>	Add a resume-aware aggregation diagnostics ledger. Round records are committed atomically so failed or resumed attempts cannot duplicate or replace good records.
<code>src/ssfl/reporting/build_report.py</code>	Produce a separate feasibility comparison keyed by run kind and aggregation mode. Extension runs cannot silently replace canonical report cells.
<code>tests/protocol/test_dawid_skene.py</code>	Add hand-computed, synthetic, missing-label, determinism, convergence, failure, and reliability tests.
Existing configuration, integration, privacy, resume, and report tests	Prove unchanged payloads, valid mode combinations, restricted audit defaults, clean resume behavior, and report separation.

8 Configuration and starting values

All settings must appear in the resolved configuration, run identity, and report. The values below are **candidate screening values**. They are not defaults claimed by the reference paper and they must be locked before active testing.

Table 3: Complete proposed configuration dictionary.

Field	Candidate	Meaning in plain English
<code>ssfl_hard_aggregation</code>	<code>dawid_skene_shadow</code>	Server aggregation mode: majority, shadow, or active Dawid-Skene.
<code>dawid_skene_max_iterations</code>	100	Hard ceiling on E/M cycles.
<code>dawid_skene_min_iterations</code>	2	Prevent a convergence declaration at the starting point.
<code>dawid_skene_tolerance</code>	10^{-6}	Relative regularized-objective change required to stop.

Field	Candidate	Meaning in plain English
dawid_skene_initialization	smoothed_votes	Starting estimate; screening also tests one-hot majority and a near-diagonal confusion start.
dawid_skene_initialization_pseudocount	0.01	Small starting mass for every class.
dawid_skene_initial_diagonal_probability	0.9	Used only in the near-diagonal initialization sensitivity arm.
dawid_skene_confusion_pseudocount	0.1	Prevent zero entries in confusion rows.
dawid_skene_class_prior_pseudocount	1.0	Prevent zero or collapsed initial class priors.
dawid_skene_min_item_annotations	1	Minimum usable client labels for a sample.
dawid_skene_min_client_annotations	1	Minimum submitted labels for a client to be fitted.
dawid_skene_min_clients	3	Practical support gate before fitting.
dawid_skene_posterior_threshold	0.0	Primary comparison does not gain by rejecting lower-confidence samples.
dawid_skene_warm_start	false	Required in the first implementation. Reject true until stable logical identity and resume semantics exist.
dawid_skene_damping	1.0	Use the full M-step update in the primary study.
dawid_skene_epsilon	10^{-12}	Log-space clipping safeguard.
dawid_skene_internal_dtype	float64	Numerical precision used by the estimator.
dawid_skene_failure_policy	majority_on_failure_or_nonconvergence	Visible continuity behavior for invalid or capped fits.
dawid_skene_save_annotations	false	Raw client-by-sample storage is off by default.
dawid_skene_full_audit_rounds	1, 10, 50	Full posterior/confusion audits for the 50-round profile; a 200-round profile can add 100, 150, and 200.

9 Feasibility and expected cost

The estimator is small enough in memory. Runtime is the main engineering uncertainty.

One full iteration has both an E-step and an M-step. The approximate contribution counts are:

- 27-client scenario: 5.29 million per full iteration, or about 529 million at 100 iterations;
- 89-client scenarios: 17.42 million per full iteration, or about 1.74 billion at 100 iterations.

The implementation must use indexed gathers, counting operations, and optional chunking rather

Table 4: Largest-scenario compact memory estimates.

Array	Shape and type	Approximate size
Annotations	$89 \times 8,900$ bytes	0.76 MiB
Posterior probabilities	$8,900 \times 11$ float64	0.75 MiB
Client confusion matrices	$89 \times 11 \times 11$ float64	0.08 MiB
Naive temporary to avoid	$89 \times 8,900 \times 11$ float64	66.5 MiB

than materializing the naive three-dimensional temporary. Early stopping should avoid using the ceiling on easy rounds. A 100-iteration live ceiling is the starting proposal; 50, 100, and 500 are screened offline. Real wall time, peak memory, and audit-writing time must be measured separately.

10 The controlled experiment

The purpose of the experiment is not to show that a complicated method can run. It is to establish whether the server-side aggregation change improves labels and downstream learning under a fair, repeatable comparison.

Table 5: Staged experiment plan. Phase 4 is not part of the primary decision.

Phase	Scale	Purpose and rule	Federated runs
0: gates	Local fixtures and scenario-sized benchmarks	Hand-computed mathematics, synthetic clients, missingness, ordering, failure, determinism, latency, and memory.	0
1: shadow	3 scenarios, seed 2023, 50 rounds	Majority stays active. Compute Dawid-Skene on the same proposals. Screen 12 declared settings on development and check one winner once on validation.	3
2: pilot	3 scenarios, seeds 2023–2025, 50 rounds	Nine matched majority/Dawid-Skene pairs. Lock settings before this phase.	18
3: confirmation	3 scenarios, seeds 2026–2035, 200 rounds	Thirty matched pairs with no setting changes after the pilot. Seeds are new and independent of screening.	60
4: later ablations	Only after confirmation	Test filtering, evidence thresholds, confidence thresholds, damping, stable-identity warm start, tie-breaks, and soft posterior distillation as separate questions.	Not in primary plan

The plan through confirmation contains 81 federated runs: 3 shadow runs, 18 pilot runs, and 60 confirmation runs. This excludes local fixtures, benchmarks, and later ablations.

10.1 Locked dataset and training controls

Control	Locked value or rule
Dataset identity	Prepared N-BaIoT manifest; hash 54023e3de197d9dc16f50a89425d5c44 bb594486210aacf16ecaa8cccc86ba65.
Split sizes	8,900 shared open samples; 62,300 private samples; 17,800 test samples.
Classes	11. Open-set counts are 900 each for classes 0–5 and 700 each for classes 6–10.
Scenarios	1, 2, and 3, with 27, 89, and 89 structural clients.
Participation	All structural clients per round, with actual participation, retries, and failures recorded.
Model and training	CNN; persistent model weights; fresh Adam per task; learning rate 10^{-4} ; five local epochs; batch size 80.
Filtering	Discriminator enabled with the median threshold in the primary comparison.
Labels	Existing hard int8 labels and abstention semantics.
Initialization	Same seeded model checkpoints inside every matched pair.
Infrastructure	Same commit, dependency lock, drivers, hardware, deterministic settings, GPU allocation, and concurrency. Run order is randomized or interleaved.
Run identity	Explicit scenario-seed pair ID, all random seeds, node-to-logical-client mapping, checkpoint state, and a clean committed worktree.
Only active difference	The server hard-label aggregation setting and its locked Dawid-Skene options.

10.2 Screening without hidden tuning freedom

The 12 candidate configurations and the exact development/validation split are declared before scores are read. Development uses scenario 1, seed 2023, rounds 1, 5, 10, 15, 20, and 25, on hash buckets 0–1 of the open items. Validation uses later scenario-1 rounds and selected rounds from scenarios 2 and 3, on hash buckets 2–4. Test accuracy from active runs is never a tuning signal.

The screen changes one setting at a time: iteration ceiling, tolerance, initialization family, initialization smoothing, confusion smoothing, or class-prior smoothing. Minimum evidence, posterior threshold, damping, and warm start remain fixed. The winner is selected by development macro-F1, then worst-class recall, runtime, lower iteration ceiling, and finally configuration ID. Any setting that fails or falls back is ineligible. If the one-time validation check misses the shadow gate, the experiment stops or a new study is preregistered; validation is not reused for retuning.

11 What will be measured

11.1 Label quality, offline only

After a run, a separate evaluator may join outputs to sealed open-set truth. Live aggregation and training cannot import those labels. The offline report includes:

- label accuracy and macro, micro, and weighted precision, recall, and F1;
- per-class confusion, recall, F1, and valid coverage;

- matched-coverage comparisons so one method cannot win merely by rejecting more samples;
- disagreements, corrected errors, newly introduced errors, and net corrections;
- tie and non-tie subsets;
- posterior negative log-likelihood, Brier score, entropy, and 15-bin equal-frequency calibration error on estimable items only;
- predicted class balance, annotation support, diagonal dominance, and estimated versus audit-only empirical client confusion.

11.2 Downstream model quality

The two co-primary outcomes are the mean server macro-F1 and mean server accuracy over the final 10 rounds. Accuracy matches the reference paper’s convention; macro-F1 protects against a model that looks good overall while failing a minority class.

Secondary outcomes include worst-class recall, per-class F1, final and best scores, learning-curve area, raw and normalized confusion matrices, loss, valid-label rate, and time to 50% and 75% accuracy. Runs that never reach a target are right-censored rather than assigned a fake round 200 achievement.

A **new class collapse** means that a supported class has Dawid-Skene final-10 recall below 5% while its matched majority run retains at least 20% recall. Every class below 5% in either arm is still disclosed.

11.3 Statistical unit

An item and a training round are not independent experiments. First calculate the Dawid-Skene-minus-majority final-10 difference inside each scenario-seed pair. Report scenarios separately. For the overall confirmation result, average the three scenario differences within each seed and treat the 10 seeds as independent blocks. Report paired intervals and the declared one-sided exact paired tests, while acknowledging the limited power of 10 blocks.

Every assigned Dawid-Skene round remains in the Dawid-Skene arm even if it falls back to majority. This is the intention-to-treat analysis. Algorithmic failures are not replaced with better seeds. An unresolved missing matched pair is disclosed and blocks adoption.

11.4 Engineering behavior

Record iterations, stop reason, raw likelihood, regularized-objective history, fallback reason, latency, peak memory, temporary allocation, audit storage, participation, and support. Confirm exact equality of network tensor names, data types, shapes, and logical byte counts. Serialized bytes are reported separately; they can vary when active training changes existing values, but no new Dawid-Skene payload is allowed.

12 Decision gates

These are proposed preregistration thresholds, not observed results or guarantees.

Table 7: Promotion, adoption, and stopping rules.

Decision	Required evidence
Shadow to active pilot	Finite, invariant-valid output every round; at least 95% convergence; no unexplained objective decrease; positive net corrections in every scenario; at least +1 percentage points matched-coverage macro-F1; at least +5 percentage points on the targeted problem-class subset; scenario-89 p95 compute time below both 10% of round time and 5 seconds; zero logical byte-count change; no new wire field; any serialized-byte difference below 1% and explained.
Pilot to confirmation	Mean paired final-10 macro-F1 at least +1 percentage points; accuracy non-inferior within -0.5 percentage points; no scenario loses more than 1 macro-F1 point; at least 8 of 9 pairs improve macro-F1; worst-class recall improves with no new class collapse; fallback below 1%; wall-clock overhead below 10%; no protocol payload change.
Adopt after confirmation	Ten seed-block macro-F1 differences average at least +1 percentage points and pass the declared one-sided test at $\alpha = 0.05$; accuracy passes the -0.5 percentage points non-inferiority test; both co-primary conditions pass; all runs are disclosed; no scenario loses more than 1 point; no new class collapse, privacy breach, protocol increase, or systems regression.
Stop or redesign	Label switching, class collapse, gains obtained mainly by rejecting samples, worse non-tie decisions, frequent sparse or numerical fallback, use of sealed labels in the live path, gains confined to one seed or scenario, unacceptable sensitive-profile risk, or excessive runtime after optimization.

13 Privacy, audit, and reproducibility

Keeping the network payload unchanged does not remove every privacy concern. Dawid-Skene can create a per-client profile of class-specific error patterns. Those profiles and the raw annotation matrix are more sensitive than aggregate vote totals.

The proposed controls are:

- raw annotation storage is off by default and allowed only for restricted Phase-1 shadow audits;
- scenario 1 retains rounds 1, 5, 10, 15, 20, 25, 30, 35, 40, 45, and 50; scenarios 2 and 3 retain rounds 1, 10, 25, 35, and 50, exactly matching the screen;
- sender IDs are replaced by a run-scoped HMAC pseudonym before storage; this is pseudonymization, not complete anonymization;
- sensitive files use compressed, encrypted, owner-only storage with directory mode 0700;
- raw annotations and the HMAC key are deleted within 30 days after Phase-1 settings are locked;
- per-client files remain outside the general report bundle;
- truth-based quality evaluation is offline and cannot feed back into an active run;

- attempt-local files are promoted atomically only after the round checkpoint commits; resume truncates rows after the checkpoint and discards incomplete attempt files.

Table 8: New or extended experiment artifacts.

Artifact	Contents and access
aggregation_metrics.parquet	Round-level mode, support, convergence, iterations, raw likelihood, objective, entropy, timing, fallback, and disagreement.
dawid_skene_clients.parquet	Restricted client reliability, support, and confusion summaries; excluded from general telemetry and report bundles.
aggregation_quality.parquet	Offline truth-based majority/Dawid-Skene comparisons; never read by training.
aggregation_audit/*.npz	Existing vote/label/mask data plus selected-round posterior, prior, confusion, and optionally restricted annotations.

Every run also records its resolved configuration, manifest hash, all seeds, scenario, client mapping, participation, retries, code revision, dependency versions, hardware, checkpoints, and dirty-worktree state. Confirmation requires a clean committed worktree.

14 Risks and limitations

Table 9: What can go wrong and how it will be detected.

Risk	Why it matters	Control or diagnostic
Clients make correlated errors	Dawid-Skene assumes errors are conditionally independent once the true class is known. Similar models can make the posterior overconfident.	Calibration, entropy, synthetic correlated-error tests, and cautious interpretation of confidence.
Abstention is not random	The discriminator is more likely to abstain on particular samples and classes, which can bias confusion estimates.	Treat abstention as missing, report class/client support, and test alternate filter modes only after the primary study.
Specialist clients lack some classes	A confusion row can have little real evidence.	Pseudocounts, minimum support, effective client-by-class support, and excluded-client reporting.
Many clients make the same mistake	Agreement without gold truth cannot expose a unanimous correlated error.	State the limitation, examine offline truth only after runs, and never claim consensus is ground truth.

Risk	Why it matters	Control or diagnostic
Hidden labels switch meaning	A latent class can move away from the repository’s semantic class index.	Vote-anchored initialization, diagonal-dominance diagnostics, synthetic tests, shadow mode, and no-go on detected switching.
Slow server rounds	Repeated E/M passes can dominate 200-round experiments.	Early stopping, iteration cap, indexed operations, chunking, separate benchmarks, and hard latency gates.
Sensitive client profiles	Confusion matrices can act like quality or behavior profiles.	Default off, pseudonymization, restricted encrypted storage, access separation, deletion, and privacy review.
Report collision	An extension could silently replace an existing canonical result.	Key reports by run kind and aggregation mode and test that canonical cells cannot be superseded.

15 Verification plan

Before any active experiment, the implementation must pass the following groups of tests.

15.1 Mathematics and estimator behavior

- one hand-derived E-step and M-step;
- perfect, unreliable, adversarial, imbalanced, sparse, and missing-label synthetic clients;
- finite and normalized priors, posteriors, and confusion rows;
- non-decreasing regularized objective within numerical tolerance;
- absent classes, all-abstain samples, exact MAP ties, and one-client inputs;
- forced numerical failure and iteration-cap fallback;
- synthetic reliability rankings and label-alignment stress tests.

15.2 Protocol, configuration, and determinism

- identical output under every valid sender order;
- deterministic handling of identical and conflicting duplicate replies;
- strict ranges and valid hard-label mode combinations;
- rejected warm start in the first implementation;
- every option changes the resolved run identity;
- canonical majority behavior remains unchanged;
- extension runs cannot populate canonical report cells.

15.3 Integration, privacy, resume, and performance

- two-round CPU smoke runs in shadow and active modes;
- shadow output and valid fallback output match majority for valid unique-sender batches;
- clients consume only the existing labels and mask;
- no new payload field or logical byte count;
- live code cannot read sealed truth labels;
- restricted files are absent by default;
- resume reproduces committed output and removes incomplete diagnostics;
- scenario-sized 27- and 89-client latency and peak-memory benchmarks.

16 Implementation sequence

If the server-side scope and experimental defaults are approved, the safest order of work is:

1. add validated configuration fields while keeping majority as the default;
2. implement the pure Dawid-Skene estimator and its tagged success/failure result;
3. add mathematical, synthetic, missing-data, and determinism unit tests;
4. add the proposal-to-annotation adapter and preserve the existing result contract;
5. add shadow and active strategy selection with explicit all-abstain no-proposal output;
6. add the resume-safe diagnostics ledger and restricted audit controls;
7. add the offline truth evaluator and report identity separation;
8. run protocol, privacy, resume, and payload-equivalence tests;
9. benchmark 27- and 89-client scenarios and optimize indexed operations if needed;
10. run Phase 1 shadow screening, lock settings, and apply the preregistered gate;
11. proceed to the paired pilot and confirmation only when each prior gate passes.

No production implementation should be called complete merely because the estimator returns labels. Configuration, failure behavior, audits, offline evaluation, reporting separation, privacy controls, and tests are part of the required change.

17 Decisions and recommended defaults

The server-only scope is now the working assumption. The remaining approvals have recommended defaults:

1. **Abstention:** keep `-1` as a missing observation. Requiring every client to label every sample would change the current protocol and filter behavior.
2. **Primary outcomes:** use final-10 macro-F1 and accuracy together. Macro-F1 protects classes; accuracy preserves comparability with the paper.

3. **Sensitive audit:** allow raw annotations only for the declared Phase-1 rounds under owner-only encrypted storage and 30-day deletion. If that storage is unavailable, compute all screen variants online and do not save raw annotations.
4. **Compute budget:** authorize one phase at a time: three shadow runs, then 18 pilot runs, then 60 confirmation runs only if the earlier gates pass.

A Plain-English server pseudocode

```

For each server round:
  collect and validate the existing hard-label proposals

  if no valid proposal exists:
    broadcast all -1 labels and an all-false valid mask
    record "no proposals"
    stop this aggregation

  if the configured mode is majority:
    run the unchanged majority aggregator
    broadcast its existing labels and mask
    stop this aggregation

  group all replies by sender
  accept one copy of identical retries
  reject a sender if its duplicate payloads conflict
  sort accepted senders and build Y[client, sample]
  treat -1 cells as missing observations

  compute the deterministic majority result for fallback

  if fewer than the required clients or items have support:
    record the support failure
    broadcast the majority result
    stop this aggregation

  initialize class probabilities from smoothed votes
  initialize the class prior and client confusion matrices

  repeat up to max_iterations:
    E-step: estimate class probabilities for every usable item
    record raw likelihood and the regularized objective
    check all finite/normalization/monotonicity rules

    if the regularized objective has converged:
      mark success and stop repeating

    if another iteration remains:
      M-step: update the class prior and client confusion matrices

  if the fit failed or hit the cap without convergence:
    record the reason
    broadcast the majority result
  else if mode is shadow:
    save Dawid-Skene diagnostics
    broadcast the majority result
  else:
    choose the highest-posterior class for each supported item

```

```
broadcast those hard labels and the existing valid mask
```

```
never broadcast posterior, confusion, or reliability arrays
commit diagnostics only after the round checkpoint commits
```

B The 12-setting shadow screen

ID	Single change from the candidate configuration
DS-00	Candidate: 100 iterations, tolerance 10^{-6} , smoothed votes, initialization pseudocount 0.01, confusion pseudocount 0.1, class-prior pseudocount 1.0.
DS-01	Maximum iterations 50.
DS-02	Maximum iterations 500.
DS-03	Tolerance 10^{-4} .
DS-04	Tolerance 10^{-8} .
DS-05	One-hot majority initialization.
DS-06	Near-diagonal confusion initialization with diagonal probability 0.9.
DS-07	Initialization pseudocount 0.001.
DS-08	Initialization pseudocount 0.1.
DS-09	Confusion pseudocount 0.01.
DS-10	Confusion pseudocount 1.0.
DS-11	Class-prior pseudocount 0.1.

All 12 keep minimum item annotations 1, minimum client annotations 1, minimum clients 3, posterior threshold 0.0, damping 1.0, and warm start false. Every attempted result is published.

C Glossary

Term	Meaning
Abstention	A client provides no label for one sample. Stored as -1 and treated as missing.
Annotation	One client's proposed label for one shared sample.
Class prior	The estimated share of usable samples belonging to each class.
Confusion matrix	A class-by-class table describing which labels a client tends to output for each hidden true class.
Convergence	The iterative estimates have changed by no more than the declared tolerance.
Damping	Blending a new update with the previous update instead of replacing it fully.
E-step	Re-estimate sample class probabilities using current client error patterns.
Fallback	Use deterministic majority when Dawid-Skene cannot finish safely.

Term	Meaning
Hard label	One selected class number rather than a full probability vector.
Latent label	The unknown class that the server is trying to estimate.
Log space	A numerically stable way to add log probabilities instead of multiplying tiny numbers.
M-step	Re-estimate the class prior and client confusion matrices using current sample probabilities.
Macro-F1	Calculate F1 for every class and average the classes equally.
Posterior probability	The estimated probability of a class after considering the submitted labels.
Pseudocount	A small artificial count that prevents zero probabilities and stabilizes sparse rows.
Shadow mode	Compute and record Dawid-Skene while continuing to broadcast majority labels.
Valid mask	A Boolean vector telling downstream code which global labels may be used.
Warm start	Start a new round from a previous round's Dawid-Skene parameters. Disabled here.

D Reference and scope note

The mathematical reference is J. Dong, R. Zhu, X. Shang, and J.-H. Xue, “Dawid-Skene-model-based label-noise mitigation for federated learning,” *Information Sciences*, volume 745, article 123425, 2026, [doi:10.1016/j.ins.2026.123425](https://doi.org/10.1016/j.ins.2026.123425).

That paper's FedDS workflow uploads client model parameters, runs the client models on public data at the server, estimates reliability, and uses reliability to weight model aggregation. This report does not propose that protocol. It proposes Dawid-Skene only at the existing server hard-label aggregation seam. Operational choices that the paper does not fully prescribe here - initialization, smoothing, abstention handling, stopping, warm start, fallback, and audit behavior - are explicit experimental settings rather than hidden assumptions.